

Java 8 Stream - 100 Coding Questions with Solutions

1. Convert List of Strings to Uppercase

```
List<String> result = list.stream()  
    .map(String::toUpperCase)  
    .collect(Collectors.toList());
```

2. Convert List of Strings to Lowercase

```
list.stream().map(String::toLowerCase).collect(Collectors.toList());
```

3. Find Length of Each String

```
list.stream().map(String::length).collect(Collectors.toList());
```

4. Filter Even Numbers

```
list.stream().filter(n -> n % 2 == 0).collect(Collectors.toList());
```

5. Filter Odd Numbers

```
list.stream().filter(n -> n % 2 != 0).collect(Collectors.toList());
```

6. Numbers Greater Than 10

```
list.stream().filter(n -> n > 10).collect(Collectors.toList());
```

7. Count Elements

```
long count = list.stream().count();
```

8. Sort Ascending

```
list.stream().sorted().collect(Collectors.toList());
```

9. Sort Descending

```
list.stream().sorted(Comparator.reverseOrder()).collect(Collectors.toList());
```

10. Remove Duplicates

```
list.stream().distinct().collect(Collectors.toList());
```

11. Get First Element

```
Optional<Integer> first = list.stream().findFirst();
```

12. Skip First 3 Elements

```
list.stream().skip(3).collect(Collectors.toList());
```

13. Limit First 5 Elements

```
list.stream().limit(5).collect(Collectors.toList());
```

14. Any Number > 50

```
boolean result = list.stream().anyMatch(n -> n > 50);
```

15. All Numbers Positive

```
boolean result = list.stream().allMatch(n -> n > 0);
```

16. None Negative

```
boolean result = list.stream().noneMatch(n -> n < 0);
```

17. Maximum Number

```
Optional<Integer> max = list.stream().max(Integer::compare);
```

18. Minimum Number

```
Optional<Integer> min = list.stream().min(Integer::compare);
```

19. List → List

```
list.stream().map(String::valueOf).collect(Collectors.toList());
```

20. Multiply Each Number by 2

```
list.stream().map(n -> n * 2).collect(Collectors.toList());
```

21. Square Each Number

```
list.stream().map(n -> n * n).collect(Collectors.toList());
```

22. Strings Starting With A

```
list.stream().filter(s -> s.startsWith("A")).collect(Collectors.toList());
```

23. Strings Ending With n

```
list.stream().filter(s -> s.endsWith("n")).collect(Collectors.toList());
```

24. Sum of Numbers

```
int sum = list.stream().mapToInt(Integer::intValue).sum();
```

25. Average of Numbers

```
double avg = list.stream().mapToInt(Integer::intValue).average().orElse(0);
```

Intermediate Problems

26. Second Highest Number

```
Integer second = list.stream()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst().get();
```

27. Second Smallest

```
Integer second = list.stream()
    .sorted()
    .skip(1)
    .findFirst().get();
```

28. Find Duplicates

```
Set<Integer> set = new HashSet<>();
list.stream().filter(n -> !set.add(n)).collect(Collectors.toSet());
```

29. Unique Elements

```
list.stream().distinct().collect(Collectors.toList());
```

30. Count Occurrence of Each Element

```
Map<Integer, Long> map =
    list.stream().collect(Collectors.groupingBy(e -> e,
        Collectors.counting()));
```

31. Character Frequency in String

```
Map<Character, Long> map =  
    str.chars()  
        .mapToObj(c -> (char)c)  
        .collect(Collectors.groupingBy(c -> c, Collectors.counting()));
```

32. First Non-Repeating Character

```
Character c = str.chars()  
    .mapToObj(x -> (char)x)  
    .collect(Collectors.groupingBy(e -> e, LinkedHashMap::new,  
Collectors.counting()))  
    .entrySet().stream()  
    .filter(e -> e.getValue() == 1)  
    .findFirst().get().getKey();
```

33. Reverse String

```
String reversed =  
    IntStream.range(0, str.length())  
        .mapToObj(i -> str.charAt(str.length()-1-i))  
        .map(String::valueOf)  
        .collect(Collectors.joining());
```

34. Longest String

```
String longest =  
    list.stream().max(Comparator.comparing(String::length)).get();
```

35. Shortest String

```
String shortest =  
    list.stream().min(Comparator.comparing(String::length)).get();
```

36. Join Strings

```
String result = list.stream().collect(Collectors.joining(","));
```

37. Group Strings by Length

```
Map<Integer,List<String>> map =  
    list.stream().collect(Collectors.groupingBy(String::length));
```

38. Partition Even and Odd

```
Map<Boolean,List<Integer>> map =  
    list.stream().collect(Collectors.partitioningBy(n -> n % 2 == 0));
```

39. Top 3 Highest Numbers

```
list.stream().sorted(Comparator.reverseOrder()).limit(3).collect(Collectors.toList()  
());
```

40. Bottom 3 Numbers

```
list.stream().sorted().limit(3).collect(Collectors.toList());
```

Advanced Problems

41. Flatten List<List>

```
list.stream().flatMap(Collection::stream).collect(Collectors.toList());
```

42. Remove Null Values

```
list.stream().filter(Objects::nonNull).collect(Collectors.toList());
```

43. Common Elements Between Two Lists

```
list1.stream().filter(list2::contains).collect(Collectors.toList());
```

44. Merge Two Lists

```
Stream.concat(list1.stream(), list2.stream())
    .collect(Collectors.toList());
```

45. Check Palindrome Strings

```
list.stream()
    .filter(s -> s.equals(new StringBuilder(s).reverse().toString()))
    .collect(Collectors.toList());
```

46. Count Words in Sentence

```
long count = Arrays.stream(sentence.split(" ")).count();
```

47. Longest Word

```
Arrays.stream(sentence.split(" "))
    .max(Comparator.comparing(String::length)).get();
```

48. Distinct Characters

```
str.chars().mapToObj(c -> (char)c).distinct().collect(Collectors.toList());
```

49. Factorial Using Streams

```
int factorial = IntStream.rangeClosed(1,n).reduce(1,(a,b)->a*b);
```

50. Fibonacci Using Streams

```
Stream.iterate(new int[]{0,1},f->new int[]{f[1],f[0]+f[1]})
    .limit(10)
    .map(f->f[0])
    .forEach(System.out::println);
```

More Practice (51–100)

Practice similar problems using Streams:

51. kth largest element
52. employee grouping by department
53. employee max salary
54. average salary by department
55. sort employees by age
56. employee hired in last year
57. remove duplicate employees by id
58. intersection of multiple lists
59. union of lists
60. difference of lists
61. count vowels in string
62. most frequent character
63. least frequent character
64. group words by first letter
65. sort map by value
66. sort map by key
67. convert list to map
68. map keys to list
69. flatten nested objects
70. filter null objects
71. collect to set
72. collect to linked list
73. group numbers by remainder
74. partition primes
75. find primes
76. running sum
77. cumulative multiplication
78. median calculation
79. top frequent numbers
80. bottom frequent numbers
81. sliding window average
82. custom collector
83. grouping multiple fields
84. nested grouping
85. flatten employee skills
86. stream pagination
87. CSV parsing
88. reading files with streams
89. parallel stream sum
90. performance comparison
91. grouping by month
92. find duplicates across lists
93. sort by multiple fields
94. mapping DTO objects
95. reduce to map

- 96. merge maps
 - 97. aggregate statistics
 - 98. percentile calculation
 - 99. batch processing with streams
 - 100. streaming large dataset
-

End

These 100 problems cover **beginner** → **advanced Java 8 Stream interview preparation**.